GL Workflow Pipeline Architecture

Technical Blueprint, Relational Metadata Schema & Airflow TaskFlow Integration

1. SYSTEM ARCHITECTURE OVERVIEW

The GL Workflow Builder is a metadata-driven pipeline generation platform designed to allow dynamic creation, editing, and orchestration of data pipelines for source systems (e.g., `fxo`, `power`). The architecture decouples UI configuration from execution mechanics through normalized SQL relational tables, enabling runtime execution via Airflow 3.x DAG factories.

Core Architectural Capabilities

- **Drag-and-Drop Canvas:** Modular composition of workflow pipelines using standard component libraries.
- **Component Types:** Supports `data_transfer`, `exclusion_rules`, `control`, `logical_mapping`, and `pipeline` types.
- **Hierarchical Nesting:** Child pipelines can be embedded as component blocks within parent pipelines recursively.
- **Phased Function Lifecycle:** Each component groups granular functions executing across three phases: PRE → `MAIN` (Null) → `POST`.
- **Intra-Component Branching:** Conditional routing and failure fallback support directly inside component function chains.
- **Field Transformations:** Explicit column-to-column mappings with transformation functions (e.g., `to_upper`, date parsing).

2. RELATIONAL DATABASE SCHEMA

The metadata layer uses six normalized relational tables to store pipeline topologies, function priorities, and field transformations.

Table Name	Primary Purpose	Key Columns
`pipelines`	Master catalog of workflow pipelines	`pipeline_id`, `pipeline_name`, `source_system`, `is_active`
`pipeline_components`	Ordered component blocks and child pipeline pointers	`component_instance_id`, `pipeline_id`, `component_type`, `nested_pipeline_id`
`component_functions`	Function execution steps with phases and trigger rules	`function_instance_id`, `phase`, `priority`, `run_in_parallel`, `trigger_rule`
`function_edges`	Intra-component task dependency graph and branching	`edge_id`, `source_function_id`, `target_function_id`, `condition_type`
`function_table_mappings`	Source-to-destination table pairs for MAIN tasks	`table_mapping_id`, `function_instance_id`, `source_table`, `destination_table`

Table Name	Primary Purpose	Key Columns
`function_field_mappings`	Column-level mapping and applied transformation functions	`field_mapping_id`, `table_mapping_id`, `source_field`, `destination_field`, `transformation_function`

3. COMPLETE SQL DEFINITION

```
-- 1. Pipelines Master Table
CREATE TABLE pipelines (
    pipeline_id INT PRIMARY KEY,
    pipeline_name VARCHAR(150) NOT NULL,
    source_system VARCHAR(50) NOT NULL,
    description VARCHAR(500),
    is_active CHAR(1) DEFAULT 'Y'
);

-- 2. Pipeline Components (Supports nested child pipelines)
CREATE TABLE pipeline_components (
    component_instance_id INT PRIMARY KEY,
    pipeline_id INT NOT NULL,
    component_type VARCHAR(50) NOT NULL,
    component_name VARCHAR(150) NOT NULL,
    sequence_number INT NOT NULL,
    nested_pipeline_id INT NULL,
    FOREIGN KEY (pipeline_id) REFERENCES pipelines(pipeline_id),
    FOREIGN KEY (nested_pipeline_id) REFERENCES pipelines(pipeline_id)
);

-- 3. Component Functions (Phase-based execution steps)
CREATE TABLE component_functions (
    function_instance_id INT PRIMARY KEY,
    component_instance_id INT NOT NULL,
    phase VARCHAR(20) NOT NULL, -- 'pre', 'main', 'post'
    function_id VARCHAR(50) NOT NULL,
    function_name VARCHAR(150) NOT NULL,
    sequence_number INT NOT NULL,
    priority INT DEFAULT 0,
    run_in_parallel CHAR(1) DEFAULT 'N',
    trigger_rule VARCHAR(50) DEFAULT 'all_success',
    is_active CHAR(1) DEFAULT 'Y',
    FOREIGN KEY (component_instance_id) REFERENCES pipeline_components(component_instance_id)
);
```

4. ADDITIONAL SQL SCHEMA (EDGES & MAPPINGS)

```
-- 4. Function Routing Edges (Intra-component branching & error paths)
CREATE TABLE function_edges (
    edge_id INT PRIMARY KEY,
    component_instance_id INT NOT NULL,
    source_function_id VARCHAR(50) NOT NULL,
    target_function_id VARCHAR(50) NOT NULL,
    condition_type VARCHAR(30) DEFAULT 'on_success', -- 'on_success', 'on_failure', 'always'
    FOREIGN KEY (component_instance_id) REFERENCES pipeline_components(component_instance_id)
);

-- 5. Table Mapping Definitions
CREATE TABLE function_table_mappings (
    table_mapping_id INT PRIMARY KEY,
    function_instance_id INT NOT NULL,
    source_table VARCHAR(150) NOT NULL,
    destination_table VARCHAR(150) NOT NULL,
    FOREIGN KEY (function_instance_id) REFERENCES component_functions(function_instance_id)
);

-- 6. Field Level Transformations
CREATE TABLE function_field_mappings (
    field_mapping_id INT PRIMARY KEY,
    table_mapping_id INT NOT NULL,
    source_field VARCHAR(150) NOT NULL,
    destination_field VARCHAR(150) NOT NULL,
    transformation_function VARCHAR(100) NULL,
    sequence_number INT NOT NULL,
    FOREIGN KEY (table_mapping_id) REFERENCES function_table_mappings(table_mapping_id)
);
```

5. AIRFLOW 3.X TASKFLOW API DYNAMIC DAG FACTORY

The following Python implementation reads pipeline metadata and constructs DAGs using Airflow's TaskFlow API, handling phase dependencies, parallel execution, and task-level failure routing.

```
from airflow.decorators import dag, task
from airflow.utils.trigger_rule import TriggerRule
from datetime import datetime

# Function Registry lookup placeholder
def execute_registered_function(function_name, **context):
    print(f"Executing function: {function_name}")

@dag(
    dag_id="dynamic_gl_workflow_generator",
    start_date=datetime(2026, 1, 1),
    schedule=None,
    catchup=False,
)
def pipeline_builder():

    # 1. Fetch metadata from DB (Structured dict representation)
    components_meta = [
        {
            "component_instance_id": 101,
            "component_name": "Ingest_FX_Data",
            "component_type": "data_transfer",
            "functions": [
                {"function_id": "f1", "function_name": "validate_source", "phase": "pre", "tr
igger_rule": "all_success"},
                {"function_id": "f2", "function_name": "transfer_fx_tables", "phase": "main",
"trigger_rule": "all_success"},
                {"function_id": "f3", "function_name": "log_error_fallback", "phase": "main",
"trigger_rule": "one_failed"},
            ],
            "edges": [
                {"source": "transfer_fx_tables", "target": "log_error_fallback", "condition":
"on_failure"}
            ]
        }
    ]

    task_registry = {}

    for comp in components_meta:
        comp_id = comp["component_instance_id"]

        # Dynamically build function tasks
        for func in comp["functions"]:
            func_name = func["function_name"]
            t_rule = getattr(TriggerRule, func["trigger_rule"].upper(), TriggerRule.ALL_SUCCE
SS)
            task_id = f"comp_{comp_id}_{func['phase']}_{func_name}"

            @task(task_id=task_id, trigger_rule=t_rule)
            def dynamic_task(f_name=func_name, **kwargs):
                return execute_registered_function(f_name, **kwargs)

            task_registry[func_name] = dynamic_task()

        # Wire task edges and failure branches
        for edge in comp.get("edges", []):
            src, tgt = edge["source"], edge["target"]
            if src in task_registry and tgt in task_registry:
                task_registry[src] >> task_registry[tgt]
```